



Universidad del Valle de Guatemala

Facultad de Ingeniería

CC3092 — Deep Learning y Sistemas Inteligentes

Laboratorio #1

Entrenamiento de Redes Neuronales (MLP)

Nicolás Concuá

Carné: 23197

2 de agosto, 2026

Laboratorio #1 — Entrenamiento de Redes Neuronales (MLP)

Repositorio: <https://github.com/nicoCT04/Lab1DL>

1. Datos y preparación (resumen)

Dataset de Kaggle: 20,640 observaciones × 10 variables (9 features + target median_house_value en USD). La feature más correlacionada con el precio es median_income (~0.69). Preparación, respetando que el test no influye en ninguna decisión:

- Split 70/15/15 → train 14,448 · val 3,096 · test 3,096.
- Nulos: solo total_bedrooms (207); imputados con la mediana del train. 0 duplicados.
- Outliers/censura: features de conteo muy sesgadas (skew 3.4–4.9); el target está censurado en \$500,001 (965 filas, 4.7%). No se eliminan; se controla con escalado.
- Categórica ocean_proximity → One-Hot (13 features finales).
- Escalado: StandardScaler (media 0, desv 1) ajustado solo con train, en features y en el target (se invierte para reportar métricas en USD).

2. Investigación: capas y optimizadores

Capas `torch.nn`:

- `nn.Linear` — capa densa $y = xW^T + b$, bloque base del MLP. Params: in_features, out_features, bias.
- `nn.ReLU` max(0,x) — barata, converge rápido; riesgo de *dying neurons*. `nn.LeakyReLU` — pendiente α (~0.01) para negativos, evita neuronas muertas. `nn.Tanh` — salida (-1,1) centrada en 0, satura en los extremos.
- `nn.Dropout` — apaga una fracción p de neuronas en entrenamiento; regulariza reduciendo el overfitting.
- `nn.BatchNorm1d` — normaliza las activaciones por *batch* (num_features); estabiliza/acelera el entrenamiento y regulariza levemente.

Pérdidas de regresión: `MSELoss` $(\hat{y}-y)^2$, penaliza fuerte los errores grandes (sensible a outliers; su raíz es el RMSE). `L1Loss` $|\hat{y}-y|$, robusta a outliers. `SmoothL1Loss` (Huber), híbrida: cuadrática cerca de 0 y lineal en errores grandes (beta marca el umbral).

Optimizadores (`torch.optim`). El `lr` fija el tamaño del paso: muy alto diverge, muy bajo se estanca. `weight\_decay` = regularización L2 (penaliza pesos grandes). `SGD`: paso simple e igual para todos los pesos (con momentum); predecible pero sensible al lr. `Adam`: lr adaptativo por parámetro (momentos de 1er y 2º orden), convergencia rápida y robusta; opción por defecto. `RMSprop`: adapta el lr por la media móvil de gradientes²; útil con objetivos ruidosos. El optimizador traduce los gradientes del *backprop* en actualizaciones de pesos, y define la velocidad y estabilidad de la convergencia.

3. Iteraciones e hiperparámetros

Estrategia de grid parcial: baseline y luego se cambia una o dos variables a la vez para atribuir el efecto. Métricas sobre validación (USD). La mejor configuración (menor RMSE) está resaltada:

#	Arquitectura	Activ.	Optim/LR	Batch/Ep	Regulariz.	MSE (val)	MAE (val)	RMSE (val)
1	[64,32]	relu	adam/0.001	64/60	—	2,690,290,688	34,898	51,868
2	[128,64,32]	relu	adam/0.001	64/60	—	2,662,400,256	34,504	51,598
3	[32,16]	relu	adam/0.001	64/60	—	2,839,648,000	36,391	53,288
4	[128,64,32]	tanh	adam/0.001	64/60	—	2,637,513,984	34,295	51,357
5	[128,64,32]	leaky_relu	adam/0.001	64/60	—	2,639,161,856	34,643	51,373
6	[128,64,32]	relu	sgd/0.01	64/60	—	2,758,229,248	37,531	52,519
7	[128,64,32]	relu	rmsprop/0.001	64/60	—	2,644,834,816	33,534	51,428
8	[128,64,32]	relu	adam/0.01	64/60	—	2,800,043,008	35,807	52,915
9	[128,64,32]	relu	adam/0.0001	64/60	—	2,779,448,832	35,790	52,720
10	[128,64,32]	relu	adam/0.001	64/60	L2=1e-4	2,693,670,912	35,419	51,901
11	[128,64,32]	relu	adam/0.001	64/60	drop=0.3	2,706,255,360	35,883	52,022
12	[128,64,32]	relu	adam/0.001	64/60	L1=1e-5	2,613,474,304	34,513	51,122
13	[128,64,32]	relu	adam/0.001	256/60	—	2,648,952,576	35,062	51,468
14	[128,64,32]	relu	adam/0.001	64/100	L2+drop0.2+BN	2,597,420,288	34,503	50,965

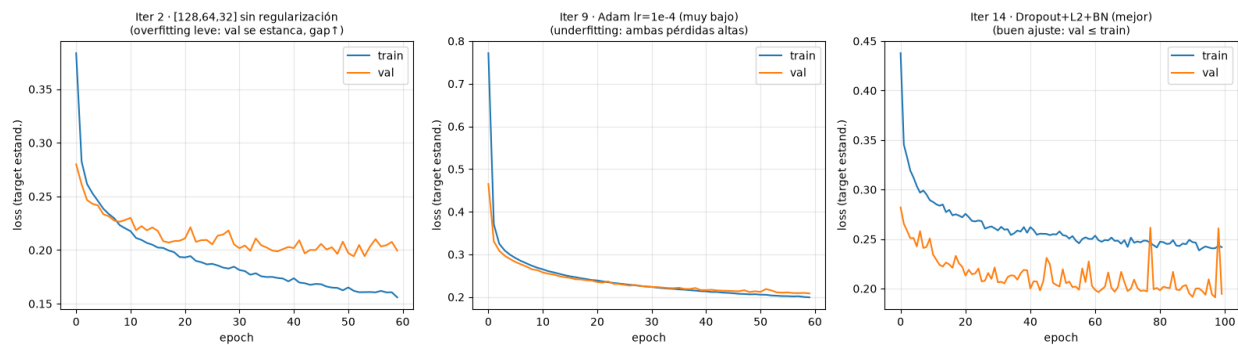


Figura 1. Curvas de pérdida de 3 iteraciones: overfitting (iter 2), underfitting (iter 9) y buen ajuste (iter 14).

4. Análisis y discusión

Mayor impacto positivo / negativo. El mayor impacto positivo lo dio la regularización combinada (iter #14: dropout+L2+BatchNorm+100 epochs), mejor RMSE de validación (\$50,965). Entre cambios individuales, la regularización L1 (#12, \$51,122) y aumentar la profundidad ([64,32]→[128,64,32]) fueron los que más ayudaron. El mayor impacto negativo fue reducir la capacidad a [32,16] (#3, \$53,288, +\$1,690) y un learning rate mal ajustado (1e-2 demasiado alto, #8, \$52,915; 1e-4 demasiado bajo, #9, \$52,720): el lr fue el hiperparámetro más sensible.

Overfitting / underfitting. Sí se observaron ambos, identificados por el gap train-val y las curvas. Las redes profundas sin regularizar (#2, #5, #7) mostraron *overfitting leve* (train baja a ~0.155 mientras val se estanca en ~0.198). Hubo *underfitting* en la red pequeña (#3) y con lr=1e-4 (#9), donde ambas pérdidas quedaron altas. Se mitigó con regularización: en #14 el gap se cierra (val ≤ train, porque dropout/BN solo actúan en entrenamiento).

¿La regularización mejoró validación? De forma modesta y dependiente de la combinación. Aplicadas solas, L2 (#10) y dropout (#11) no mejoraron respecto a la referencia (incluso un poco peor), porque el modelo base no sufría un overfitting severo. La que mejor funcionó individualmente fue L1 (#12): al empujar pesos a cero, hace una selección de features útil frente a columnas poco informativas (p. ej. la dummy ISLAND, con solo 5 casos). La mejor configuración global combinó dropout+L2+BatchNorm con más epochs, que juntos sí superaron a todas.

Batch size y epochs. El batch grande (256, #13) entrenó más rápido (~2 s vs ~5 s) y con curvas más estables/suaves (menos ruido por actualización), a métrica prácticamente igual; el batch pequeño da gradientes más ruidosos. Más epochs (100 en #14) fueron necesarias porque la regularización frena la convergencia: el modelo regularizado necesita más pasos para alcanzar su mínimo. En general: batch grande → estable pero requiere más epochs/lr ajustado; más epochs → mejor convergencia, con riesgo de overfitting si no se regulariza.

MSE vs MAE vs RMSE. El MAE (~\$34.5k) es bastante menor que el RMSE (~\$51k). Como MAE es el error típico y RMSE penaliza al cuadrado, esa brecha revela que existen errores grandes puntuales (viviendas censuradas en \$500k y outliers) que inflan el RMSE/MSE pero no tanto el MAE. El MSE (en USD²) no es interpretable directamente; el RMSE lo devuelve a dólares y castiga los fallos grandes; el MAE describe el error del caso promedio. Juntas indican una distribución de error con cola pesada.

Modelo de producción. Elegiría la arquitectura `[128,64,32]` con ReLU, Adam (lr≈1e-3), BatchNorm + dropout moderado (0.2) + L2 (1e-4) y early stopping sobre validación (~80–100 epochs). Para seguir optimizando usaría búsqueda aleatoria para una exploración amplia inicial y luego optimización bayesiana para refinar: es más eficiente en número de entrenamientos que grid search (que crece de forma combinatoria) y aprovecha los resultados previos para proponer configuraciones prometedoras, algo valioso cuando cada entrenamiento es costoso.

5. Conclusiones

- El MLP final alcanza RMSE ≈ \$52,814 y MAE ≈ \$36,110 en test, muy cerca de validación (RMSE \$50,965) → generaliza bien, sin sobreajuste al conjunto de validación.
- Los factores de mayor impacto fueron el learning rate y la capacidad de la red; la regularización aportó mejoras modestas y solo la combinación (dropout+L2+BN+epochs) superó claramente al baseline.
- La censura del target en \$500k pone un piso al error alcanzable e infla el RMSE frente al MAE; un modelo productivo debería tratar ese rango de forma especial.